

這是一個專為開發者設計、零額外依賴（輕量級）的最小可行實作專案 **mini-okf-agent**。

你可以直接建立以下目錄與檔案，這套程式碼展示了如何將 **OKF Knowledge Bundle**、**雙軌圖譜檢索 (Vector + Graph Traversal)**、**Prompt Caching 機制**與 **LLM 推論** 完整串接起來。

專案目錄結構

```
mini-okf-agent/  
├── knowledge_bundle/           # OKF 知識庫  
│   ├── datasets/              # 概念 1：訂單資料表 Schema  
│   │   └── orders.md  
│   ├── metrics/               # 概念 2：ARR 計算邏輯（連結至 orders.md）  
│   │   └── arr.md  
│   └── runbooks/              # 概念 3：支付失敗 SOP（連結至 orders.md）  
│       └── handle-failed-payout.md  
├── okf_engine.py              # OKF 解析、圖譜建立與 Context Gateway  
├── main.py                    # 專案入口點（展示檢索與 Prompt 組裝）  
└── requirements.txt           # 專案依賴套件
```

Step 1: 建立依賴檔案 **requirements.txt**

```
python-frontmatter>=1.0.0  
pydantic>=2.0.0
```

（註：這個範例使用內建語意搜尋模擬 *Vector DB*，讓你可以直接在本地零設定運行。）

Step 2: 建立 OKF 知識庫檔案 (Knowledge Bundle)

檔案一：**knowledge_bundle/datasets/orders.md**

```
---  
type: dataset  
title: Customer Orders Table  
description: Core transactional table storing order status and payment  
totals.  
tags:  
  - finance  
  - database  
---  
  
# Customer Orders Table
```

Main table for tracking customer checkout events.

Key Schema

- * `order\_id`: String (Primary Key)
- * `customer\_id`: String
- * `status`: Enum ('PENDING', 'COMPLETED', 'FAILED')
- * `total\_usd`: Decimal

檔案二：knowledge_bundle/metrics/arr.md

```
---
type: metric
title: Annual Recurring Revenue (ARR)
description: Calculation logic for annual recurring revenue from completed
subscriptions.
tags:
  - finance
  - analytics
---
```

Annual Recurring Revenue (ARR)

ARR measures the predictable revenue generated by active subscriptions annually.

Calculation Formula

Sum of `total\_usd` where `status = 'COMPLETED'` over a 12-month period in the [Orders Dataset](../datasets/orders.md).

檔案三：knowledge_bundle/runbooks/handle-failed-payout.md

```
---
type: runbook
title: Failed Payout Recovery Procedure
description: SOP for engineering team when order payment processing fails.
tags:
  - ops
  - runbook
---
```

Failed Payout Recovery Procedure

Follow these steps if a customer reports a charged order stuck in `FAILED` state.

SOP Steps

1. Query the [Customer Orders Table](../datasets/orders.md) using `order\_id`.
2. Check if `status` is marked as `FAILED`.
3. Re-trigger payment gateway webhook or execute manual refund.

Step 3: 撰寫核心 Engine `okf_engine.py`

此檔案負責：

1. 解析 **OKF** (提取 Frontmatter & Markdown 正文)。
2. 建立 **Cross-link** 圖譜 (追蹤相對路徑 `[text](../path.md)`)。
3. **Context Gateway** (模擬 Vector DB 找到起點 + 圖譜展開 1-Hop 關聯文件)。

```
import os
import re
import frontmatter
from typing import Dict, List, Set
from pydantic import BaseModel, Field

class OKFFrontmatter(BaseModel):
    type: str
    title: str | None = None
    description: str | None = None
    tags: List[str] = Field(default_factory=list)

class OKFKnowledgeEngine:
    def __init__(self, bundle_root: str):
        self.bundle_root = bundle_root
        self.concepts: Dict[str, dict] = {} # key: relative_path
        self.graph: Dict[str, Set[str]] = {} # key: relative_path ->
        value: set of linked rel_paths
        self._load_bundle()

    def _load_bundle(self):
        """掃描 Bundle 目錄，提取 Frontmatter 並建立連結圖譜"""
        for root, _, files in os.walk(self.bundle_root):
            for file in files:
                if file.endswith(".md") and file not in ["log.md",
"okf.yaml"]:
                    abs_path = os.path.join(root, file)
                    rel_path = os.path.relpath(abs_path,
self.bundle_root).replace("\\", "/")

                    post = frontmatter.load(abs_path)
                    meta = OKFFrontmatter(**post.metadata)

                    self.concepts[rel_path] = {
                        "metadata": meta.model_dump(),
                        "content": post.content,
```

```

        "title": meta.title or rel_path
    }
    self.graph[rel_path] =
self._extract_relative_links(abs_path, post.content)

def _extract_relative_links(self, current_file_abs: str, content: str)
-> Set[str]:
    """自動解析 Markdown 中的相對路徑連結"""
    links = set()
    matches = re.findall(r'\[.*?\]\((?!http)(.*?\.\md)\)', content)
    current_dir = os.path.dirname(current_file_abs)

    for link in matches:
        target_abs = os.path.normpath(os.path.join(current_dir, link))
        if os.path.exists(target_abs):
            target_rel = os.path.relpath(target_abs,
self.bundle_root).replace("\\", "/")
            links.add(target_rel)
    return links

def mock_vector_search(self, query: str) -> str:
    """模擬 Vector DB 近似搜尋：根據關鍵字找到最相關的第一個 OKF 概念檔案"""
    query_lower = query.lower()
    for path, data in self.concepts.items():
        if any(k in query_lower for k in [path, data["title"].lower(),
data["metadata"]["type"]]):
            return path
        for tag in data["metadata"]["tags"]:
            if tag in query_lower:
                return path
    # 預設 fallback
    return list(self.concepts.keys())[0]

def retrieve_context_bundle(self, user_query: str, max_hops: int = 1) -
> List[dict]:
    """Context Gateway 核心：Vector 檢索起點 + Graph 展開 1 Hop 關聯概念"""
    primary_path = self.mock_vector_search(user_query)

    visited = set()
    queue = [(primary_path, 0)]
    retrieved_concepts = []

    while queue:
        current_path, depth = queue.pop(0)
        if current_path in visited or current_path not in
self.concepts:
            continue

        visited.add(current_path)
        data = self.concepts[current_path]
        retrieved_concepts.append({
            "path": current_path,
            "metadata": data["metadata"],
            "content": data["content"]

```

```

    })

    # 圖譜走訪 (Graph Expansion)
    if depth < max_hops:
        for neighbor in self.graph.get(current_path, []):
            if neighbor not in visited:
                queue.append((neighbor, depth + 1))

    return retrieved_concepts

```

Step 4: 建立執行腳本 `main.py`

此腳本展示 Context Gateway 如何將檢索結果組裝成符合 **Prompt Caching** 規格的 Prompt，準備餵給 LLM：

```

import json
from okf_engine import OKFKnowledgeEngine

def build_prompt_with_cache(context_documents: list[dict], user_query: str)
-> dict:
    """
    組裝具有 Prompt Caching 架構的 API Payload (例如 Anthropic / Gemini API 格式)
    """
    # 組合動態 OKF 上下文區塊
    okf_context_text = "以下是從企業 OKF 知識庫檢索出的相關規範與 SOP：\n\n"
    for doc in context_documents:
        okf_context_text += f"<!-- OKF FILE: {doc['path']} (Type: {doc['metadata']['type']}) -->\n"
        okf_context_text += f"{doc['content']}\n\n---\n\n"

    # API Payload 結構
    payload = {
        "system_prompt": [
            {
                "type": "text",
                "text": "你是一個企業級 AI 助理。請嚴格依據下方提供 OKF 知識庫進行
回答。",
                "cache_control": {"type": "ephemeral"} # ← Static Cache
            },
            {
                "type": "text",
                "text": okf_context_text,
                "cache_control": {"type": "ephemeral"} # ← Dynamic OKF
            }
        ],
        "messages": [

```

```

        {
            "role": "user",
            "content": user_query # ← 非 Cache，每次傳輸算基本 Token
        }
    ]
}
return payload

if __name__ == "__main__":
    # 1. 初始化 OKF Engine
    engine = OKFKnowledgeEngine(bundle_root="./knowledge_bundle")

    print(f"[OKF Engine] 成功載入 {len(engine.concepts)} 個概念檔案。")
    print(f"[OKF Engine] 內部知識圖譜鏈結關係: {engine.graph}\n")

    # 2. 測試查詢：「如果訂單支付失敗，工程 SOP 是什麼？」
    user_query = "我們的訂單支付失敗 (failed payout) 時，工程師的處理步驟是什麼？"
    print(f"User Query: '{user_query}'\n")

    # 3. 透過 Context Gateway 檢索 (自動抓取 SOP + 連結的 Orders Table Schema)
    retrieved_docs = engine.retrieve_context_bundle(user_query, max_hops=1)

    print("=== [Context Gateway 檢索結果 (Graph Expanded)] ===")
    for doc in retrieved_docs:
        print(f" -> 載入概念檔案: {doc['path']} (Type: {doc['metadata']
['type']})")
    print("=====\n")

    # 4. 生成具備 Prompt Caching 結構的 Payload
    final_payload = build_prompt_with_cache(retrieved_docs, user_query)

    print("=== [最終組裝成的 Prompt Caching API Payload] ===")
    print(json.dumps(final_payload, indent=2, ensure_ascii=False))

```

如何執行與測試？

1. 安裝套件：

```
pip install -r requirements.txt
```

2. 執行主程式：

```
python main.py
```

執行的預期輸出與觀察重點：

當你執行 `main.py` 時，你會發現：

1. **雙軌檢索生效**：使用者只問了 `failed payout`，`okf_engine` 透過語意/標籤找到了起點 `runbooks/handle-failed-payout.md`。
2. **自動圖譜走訪 (Graph Expansion)**：Engine 發現 `handle-failed-payout.md` 裡面有相對連結 `[Customer Orders Table](../datasets/orders.md)`，於是自動把 `orders.md` 也拉進 Context 中。
3. **Prompt Cache 結構**：輸出的 JSON 顯示 OKF 檔案內容已經被放入帶有 `"cache_control": {"type": "ephemeral"}` 的 System Prompt 區塊中，直接可以送往 LLM API 享受高命中率與低成本優化。

這個小專案即為完整 OKF Agent 的核心原型，你可以直接將 `okf_engine.py` 的 `mock_vector_search` 替換為你的實際 Vector DB（如 Qdrant, Milvus, FAISS），就能輕鬆整合進你目前的專案中了！

這確實是整個架構最核心、也最精妙的地方！

OKF 文件的向量化（Vectorization）與傳統 RAG 最大的不同在於：傳統 **RAG** 是對「內文做盲目切片（Chunking）」，而 **OKF** 是對「結構化的 Semantic Unit（語意單元）」做雙重向量化。

OKF 向量化的核心技巧（兩大黃金法則）

在實作 OKF 向量化時，有兩個關鍵原則：

1. **富化文字 (Text Enrichment / Dense Context Input)**：不要只把 Markdown 內文送進 Embedding Model，而要把 **title + description + tags + content** 組合起來。這會讓 Embedding 向量同時帶有「高階語意 (Metadata)」與「詳細內容 (Body)」。
2. **語意 Payload 保留 (Metadata Preservation)**：把 YAML Frontmatter 裡的所有欄位（如 `type`, `tags`），直接寫入 Vector Database 的 Payload / Metadata。這能讓系統隨時執行 **Pre-filtering (Metadata 過濾)**。

可運行的具體實作程式碼

我們使用 Python 最常見的 `sentence-transformers`（生成 Vector）與記憶體向量索引庫（模擬 ChromaDB / Qdrant / Milvus 的運作）來重構原有的 `OKFKnowledgeEngine`。

1. 安裝必要套件

```
pip install python-frontmatter pydantic sentence-transformers numpy
```

2. 重構後的 `okf_vector_engine.py`

此程式碼展示了：

1. 如何將 OKF 文件組裝並產生 Embeddings。

2. 如何儲存至 **Vector DB** (含 **Metadata/Payload**)。
3. 如何進行真正的高維度 **Cosine Similarity** 向量檢索。

```
import os
import re
import frontmatter
import numpy as np
from typing import Dict, List, Set
from pydantic import BaseModel, Field
from sentence-transformers import SentenceTransformer

class OKFFrontmatter(BaseModel):
    type: str
    title: str | None = None
    description: str | None = None
    tags: List[str] = Field(default_factory=list)

class OKFVectorEngine:
    def __init__(self, bundle_root: str, model_name: str = "all-MiniLM-L6-
v2"):
        self.bundle_root = bundle_root

        # 1. 初始化 Embedding 模型 (實務上可換成 OpenAI / Cohere / BGE 等)
        print(f"[Vector Engine] 載入 Embedding 模型 ({model_name})...")
        self.encoder = SentenceTransformer(model_name)

        # 2. 儲存資料結構
        self.concepts: Dict[str, dict] = {}          # key: relative_path
        self.graph: Dict[str, Set[str]] = {}          # Adjacency list for
links

        # 3. 模擬 Vector Database 的內存 Index
        self.vector_store: List[dict] = []            # 存放 {path, embedding,
payload}

        # 4. 執行載入與向量化
        self._load_and_vectorize_bundle()

    def _prepare_text_for_embedding(self, meta: OKFFrontmatter, content:
str) -> str:
        """
        【OKF 向量化核心】：組合 Metadata 與 Markdown 正文，形成語意極度豐富的向量
輸入字串
        """
        title_str = meta.title or "Untitled"
        desc_str = meta.description or ""
        tags_str = ", ".join(meta.tags) if meta.tags else "None"

        # 將 OKF 的結構化資訊前置，大幅提升向量檢索精準度
        enriched_text = (
            f"Document Title: {title_str}\n"
            f"Concept Type: {meta.type}\n"
```



```

        f"Description: {desc_str}\n"
        f"Tags: {tags_str}\n\n"
        f"Document Content:\n{content}"
    )
    return enriched_text

def _load_and_vectorize_bundle(self):
    """掃描 Bundle、解析 Frontmatter、計算向量並建立 Index"""
    texts_to_embed = []
    paths_in_order = []

    print("[Vector Engine] 開始掃描並解析 OKF Bundle...")
    for root, _, files in os.walk(self.bundle_root):
        for file in files:
            if file.endswith(".md") and file not in ["log.md",
"okf.yaml"]:
                abs_path = os.path.join(root, file)
                rel_path = os.path.relpath(abs_path,
self.bundle_root).replace("\\", "/")

                post = frontmatter.load(abs_path)
                meta = OKFFrontmatter(**post.metadata)

                # A. 保存 Raw Data
                self.concepts[rel_path] = {
                    "metadata": meta.model_dump(),
                    "content": post.content,
                    "title": meta.title or rel_path
                }
                self.graph[rel_path] =
self._extract_relative_links(abs_path, post.content)

                # B. 組裝富化字串並準備進行向量化
                enriched_text = self._prepare_text_for_embedding(meta,
post.content)

                texts_to_embed.append(enriched_text)
                paths_in_order.append(rel_path)

            # C. 批量轉換為向量 (Batch Embedding)
            print(f"[Vector Engine] 正在將 {len(texts_to_embed)} 個 OKF 文件向量
            化...")
            embeddings = self.encoder.encode(texts_to_embed,
            normalize_embeddings=True)

            # D. 寫入 Vector Database (向量 + Payload)
            for rel_path, embedding in zip(paths_in_order, embeddings):
                self.vector_store.append({
                    "path": rel_path,
                    "embedding": embedding,
                    "payload": self.concepts[rel_path]["metadata"] # Vector DB
            Payload!
                })
            print(f"[Vector Engine] 向量資料庫索引建立完成！\n")

```

```

def _extract_relative_links(self, current_file_abs: str, content: str)
-> Set[str]:
    links = set()
    matches = re.findall(r'\[.*?\]\(((?!http)(.*?\.\md)\))', content)
    current_dir = os.path.dirname(current_file_abs)

    for link in matches:
        target_abs = os.path.normpath(os.path.join(current_dir, link))
        if os.path.exists(target_abs):
            target_rel = os.path.relpath(target_abs,
self.bundle_root).replace("\\", "/")
            links.add(target_rel)
    return links

def vector_search(self, query: str, filter_type: str = None, top_k: int
= 1) -> List[Tuple[str, float]]:
    """
    真正的向量餘弦相似度檢索 (Cosine Similarity Search) + Metadata Pre-
filtering
    """
    # 1. 將 User Query 轉換為向量
    query_vector = self.encoder.encode(query,
normalize_embeddings=True)

    scores = []
    for item in self.vector_store:
        # Metadata Filtering (例如只找 type == 'runbook' 的 OKF 文件)
        if filter_type and item["payload"].get("type") != filter_type:
            continue

        # 2. 計算餘弦相似度 (Dot product of normalized vectors)
        similarity = float(np.dot(query_vector, item["embedding"]))
        scores.append((item["path"], similarity))

    # 3. 排序並回傳 Top-K 相似度的 OKF 檔案路徑
    scores.sort(key=lambda x: x[1], reverse=True)
    return scores[:top_k]

def retrieve_context_bundle(self, user_query: str, max_hops: int = 1) -
> List[dict]:
    """
    完全體 Context Gateway :
    1. 透過 Vector Search 找到語意最相符的起點 OKF (Primary Node)
    2. 沿著 OKF 鏈結圖譜 (Graph) 展開 1~2 Hops 自動補齊關聯知識
    """
    search_results = self.vector_search(user_query, top_k=1)
    if not search_results:
        return []

    primary_path, score = search_results[0]
    print(f"[Vector Search] 找到最高相似度 OKF 起點: '{primary_path}'
(Cosine Score: {score:.4f})")

    visited = set()

```

```

queue = [(primary_path, 0)]
retrieved_concepts = []

while queue:
    current_path, depth = queue.pop(0)
    if current_path in visited or current_path not in
self.concepts:
        continue

    visited.add(current_path)
    data = self.concepts[current_path]
    retrieved_concepts.append({
        "path": current_path,
        "metadata": data["metadata"],
        "content": data["content"]
    })

    # 圖譜走訪 (Graph Expansion)
    if depth < max_hops:
        for neighbor in self.graph.get(current_path, []):
            if neighbor not in visited:
                queue.append((neighbor, depth + 1))

return retrieved_concepts

# ===== 測試執行區塊 =====
if __name__ == "__main__":
    # 使用上一次建立的 knowledge_bundle 目錄
    engine = OKFVectorEngine(bundle_root="./knowledge_bundle")

    # 提問測試 (測試無直接關鍵字的語意查詢)
    user_query = "扣款失敗了，工程團隊有什麼應變標準流程嗎？"
    print(f"User Query: '{user_query}'\n")

    results = engine.retrieve_context_bundle(user_query, max_hops=1)

    print("\n=== [Context Gateway 最終傳遞給 LLM 的 OKF 上下文] ===")
    for doc in results:
        print(f" -> 文件路徑: {doc['path']} | 類型: {doc['metadata']
['type']}")

```

這個實作解決了哪些關鍵問題？

1. **不需要破壞性的盲切 (No Chunking Artifacts)**：程式中我們直接把一個 OKF 檔案轉成單一向量 (Single Embedding)。因為 OKF 每個檔案本來就定義為一個獨立概念 (Concept)，避免了傳統 RAG 把程式碼或 SOP 切成兩半的問題。
2. **向量帶有結構化標籤 (Enriched Embedding)**：`_prepare_text_for_embedding` 函式將 YAML 中的 `title` 與 `tags` 一起編碼，讓 Vector Search 對自然語言的意圖捕捉 (例如「扣款失敗 SOP」對應到 `type: runbook`) 變得極度靈敏。

3. **Payload Filter 與 Vector DB 正確串接**：`self.vector_store` 中的 `payload` 保存了 OKF YAML 的所有原生欄位，當專案升級至真正的生產級向量資料庫（如 Qdrant / Milvus）時，可以無縫調用其原生的 Pre-filtering 語法。